

Assignment 1

DATA 260 Agentic AI

Name: Kazi Samina Maraj Mumu

Email: Kazisaminamaraj.mumu@sjsu.edu

Date: 08/30/2026

Student	Kazi Samina Maraj Mumu
SID4	4098
Assigned domain	Municipal transit incidents
Repository	https://github.com/SaminaMaraj/data260-4098
Submission tag	hw1
Source checkpoint	969f82f10444a1e3a9704c106c15c817d06a189e
Prepared	August 30, 2026

This report documents a municipal transit incident web application, local Docker deployment, AWS ECS Fargate deployment, a three-agent Ollama pipeline, a 40-run non-determinism experiment, and a five-turn token-accounting client.

Verification status: PASS

1. Configuration and Reproducibility

Parameter	Value	Parameter	Value
SID4	4098	PORT_BASE	8498
PREFIX	s4098	SEED	4098
VERIFY_SEED	264098	DOMAIN_ID	2
Hardware	Apple Silicon Mac M5, 24 GB	Python	3.12.10
Local model	qwen3:8b	Container platform	linux/amd64
AWS region	us-east-1	Compute	ECS Fargate

Repository structure

- `code/web_application/`: HTML, CSS, and JavaScript form application.
- `code/agents_demo.py`: Planner, Reviewer, and Finalizer pipeline.
- `code/run_nondeterminism.py`: resumable 40-run experiment.
- `src/model_client.py` and `code/hw1_client.py`: common Ollama adapter and interactive client.
- `reports/hw01/`: run log, metrics, AI disclosure, verification, raw JSON, and screenshots.

The verification result is recorded in `reports/hw01/verification.json`. The public repository is the authoritative source for the complete code and raw data.

Domain schema

Field	Purpose	Required
incidentTitle	Short incident title	Yes
routeLine	Affected route or transit line	Yes
submitterEmail	Reporter email address	Yes
description	Detailed incident description	Yes
category	Delay, collision, suspension, or infrastructure	Yes
termsAccepted	Terms acceptance	Yes
submissionDate	Generated UTC timestamp	Generated

2. Part 1 - Web Application and Docker

The application implements the assigned municipal transit incident domain. The form collects a title, route or line, email, description, category, and terms of acceptance. JavaScript performs validation and demonstrates JSON serialization, parsing, destructuring, spread syntax, a generated timestamp, and a closure-based submission counter.

HTML form - code/web_application/index.html

```
code > web_application > <> index.html > <> html > <> head > <> link
1  <!DOCTYPE html>
2  <html lang="en">
3
4  <head>
5    <meta charset="UTF-8">
6    <meta name="viewport" content="width=device-width, initial-scale=1.0">
7    <title>HW1-Kazi Samina Maraj Mumu</title>
8    <link rel="stylesheet" href="style.css">
9  </head>
10
11 <body>
12   <h1>Municipal Transit Incident Report</h1>
13   <form id="incidentForm" novalidate>
14     <label for="incidentTitle">Incident Title</label>
15     <input type="text" id="incidentTitle" name="incidentTitle" placeholder="Enter a short incident title" required
16       autofocus>
17     <label for="routeLine">Route or Transit Line</label>
18     <input type="text" id="routeLine" name="routeLine" placeholder="Enter the affected route or transit line"
19       required>
20
21     <label for="submitterEmail">Submitter Email</label>
22     <input type="email" id="submitterEmail" name="submitterEmail" placeholder="Enter your email address" required>
23     <label for="description">Incident Description</label>
24     <textarea id="description" name="description" placeholder="Describe the transit incident in detail"
25       required></textarea>
26     <label for="category">Incident Category</label>
27     <select id="category" name="category" required>
28       <option value="" disabled selected>Choose a category</option>
29       <option value="delay">Delay</option>
30       <option value="collision">Collision</option>
31       <option value="service-suspension">Service Suspension</option>
32       <option value="infrastructure-issue">Infrastructure Issue</option>
33     </select>
34     <label>
35       <input type="checkbox" id="termsAccepted" name="termsAccepted" required>
36       I agree to the terms and conditions.
37     </label>
38     <button type="submit">Submit Transit Incident</button>
39
40   </form>
41   <script src="script.js"></script>
42 </body>
43 </html>
```

JavaScript behavior - code/web_application/[script.js](#)

```

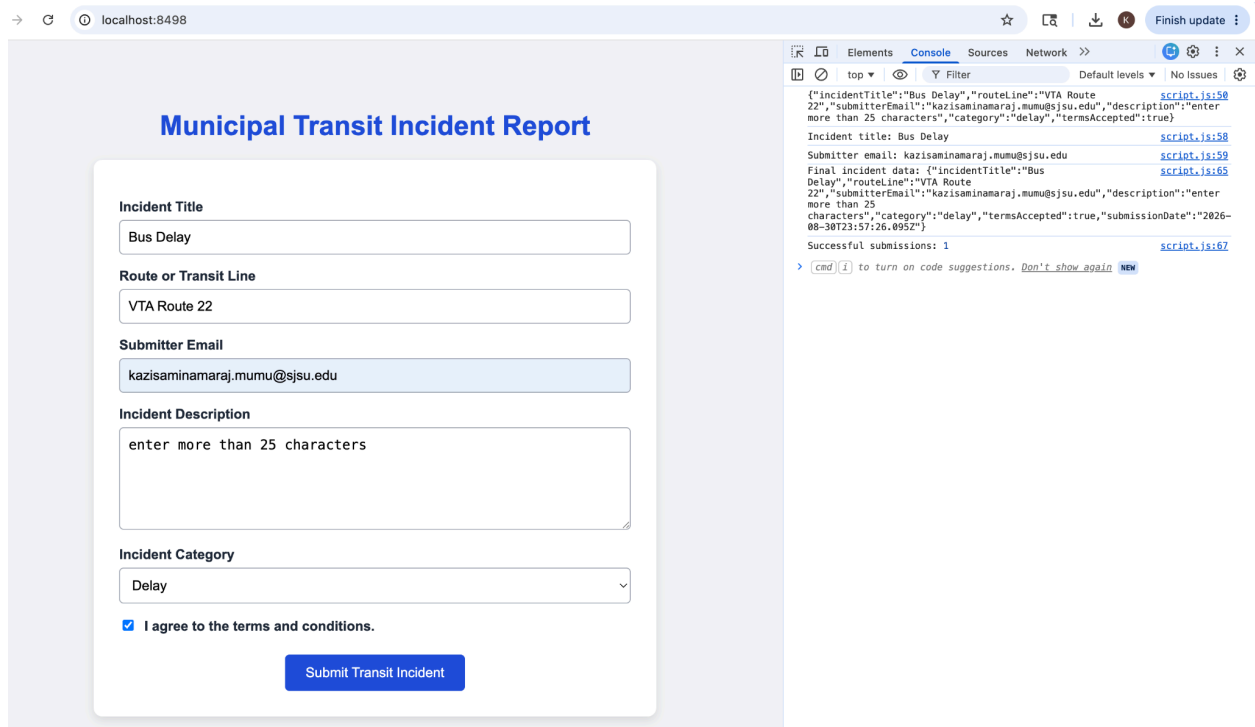
code > web_application > JS script.js > ...
1   const form = document.querySelector("#incidentForm");
2
3   const isValidDescription = (text) => text.trim().length > 25;
4   const createSubmissionCounter = () => {
5       let count = 0;
6
7       return () => {
8           count += 1;
9           return count;
10      };
11  };
12
13  const countSuccessfulSubmission = createSubmissionCounter();
14
15  form.addEventListener("submit", (event) => {
16      event.preventDefault();
17
18      const description = document.querySelector("#description").value;
19      const termsAccepted = document.querySelector("#termsAccepted").checked;
20
21      if (!isValidDescription(description)) {
22          alert("The incident description must contain more than 25 characters.");
23          return;
24      }
25
26      if (!termsAccepted) {
27          alert("You must agree to the terms and conditions.");
28          return;
29      }
30
31      if (!form.checkValidity()) {
32          form.reportValidity();
33          return;
34      }
35
36      const incidentTitle = document.querySelector("#incidentTitle").value.trim();
37      const routeLine = document.querySelector("#routeLine").value.trim();
38      const submitterEmail = document.querySelector("#submitterEmail").value.trim();
39      const category = document.querySelector("#category").value;

```

```

39
40      const incidentData = {
41          incidentTitle,
42          routeLine,
43          submitterEmail,
44          description: description.trim(),
45          category,
46          termsAccepted
47      };
48
49      const jsonString = JSON.stringify(incidentData);
50      console.log(jsonString);
51      const parsedData = JSON.parse(jsonString);
52
53      const {
54          incidentTitle: parsedIncidentTitle,
55          submitterEmail: parsedSubmitterEmail
56      } = parsedData;
57      const parsedIncidentTitle = any
58      console.log("Incident title:", parsedIncidentTitle);
59      console.log("Submitter email:", parsedSubmitterEmail);
60      const finalIncidentData = {
61          ...parsedData,
62          submissionDate: new Date().toISOString()
63      };
64
65      console.log("Final incident data:", JSON.stringify(finalIncidentData));
66      const successfulCount = countSuccessfulSubmission();
67      console.log("Successful submissions:", successfulCount);
68  });
69

```



2.1 Local Docker Evidence

Container image

code >  Dockerfile

```
1 FROM nginx:alpine
2
3 RUN rm -rf /usr/share/nginx/html/*
4
5 COPY web_application/ /usr/share/nginx/html/
6
7 EXPOSE 80
8
9 CMD ["nginx", "-g", "daemon off;"]
```

Reproducible commands

```
## Run the web application locally with Docker

Run from the repository root:

```bash
docker build --platform linux/amd64 -t data260-4098-web ./code
docker run -d --platform linux/amd64 --name data260-4098-container -p 8498:80 data260-4098-web
```

Open http://localhost:8498.

Stop the container afterward:

```bash
docker stop data260-4098-container
```
```

```
(base) samina@Kazis-MacBook-Pro-2 code % docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
4d5678c6f4f0   data260-4098-web "/docker-entrypoint..." 4 minutes ago  Up 4 minutes  0.0.0.0:8498->80/tcp, [::]:8498->80/tcp  data260-4098-container
(base) samina@Kazis-MacBook-Pro-2 code %
```

Whole Image 2644x1778 246.78KB

Figure 1. Local Docker container running with host port 8498 mapped to container port 80.

The screenshot shows a web browser window with the address bar displaying 'localhost:8498'. The main content is a form titled 'Municipal Transit Incident Report'. The form is centered on a light gray background. It contains the following fields and elements:

- Incident Title:** A text input field with the placeholder text 'Enter a short incident title'.
- Route or Transit Line:** A text input field with the placeholder text 'Enter the affected route or transit line'.
- Submitter Email:** A text input field with the placeholder text 'Enter your email address'.
- Incident Description:** A large text area with the placeholder text 'Describe the transit incident in detail'.
- Incident Category:** A dropdown menu with the text 'Choose a category' and a downward arrow.
- Terms and Conditions:** A checkbox followed by the text 'I agree to the terms and conditions.'.
- Submit Button:** A blue button with the text 'Submit Transit Incident'.

Figure 2. Municipal Transit Incident Report served locally from the Docker container.

2.2 AWS ECR and ECS Fargate Evidence

The nginx image was tagged and uploaded to Amazon ECR. An ECS cluster, Fargate task definition, and service were then created in us-east-1. The task used 0.25 vCPU, 0.5 GB memory, TCP port 80, a public IP, and security group data260-4098-web-sg allowing inbound HTTP on port 80.

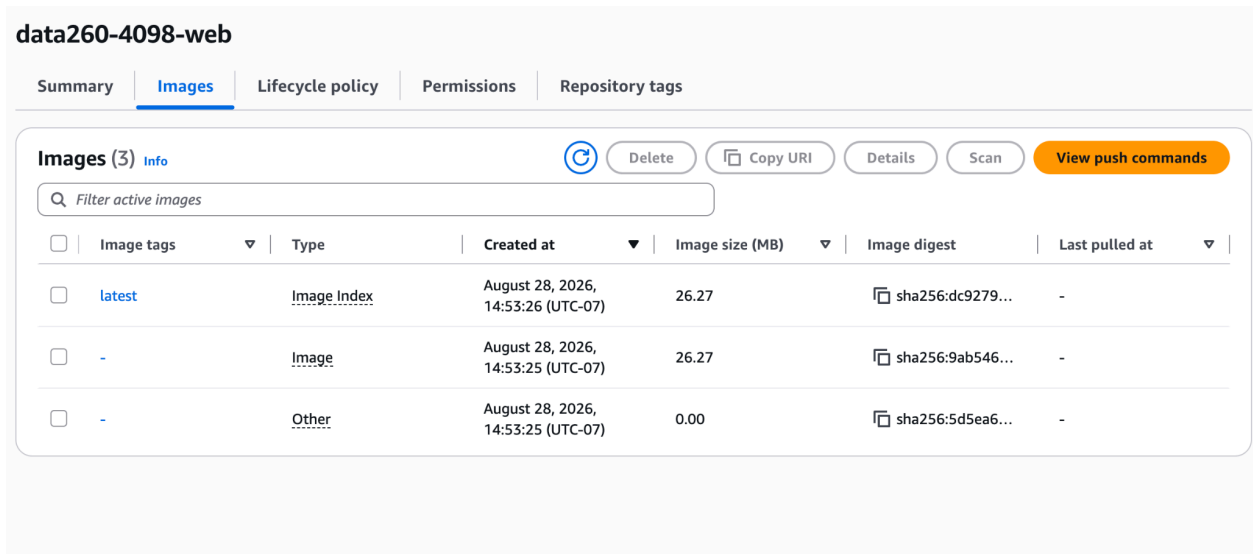


Figure 3. Container image available in the Amazon ECR repository.

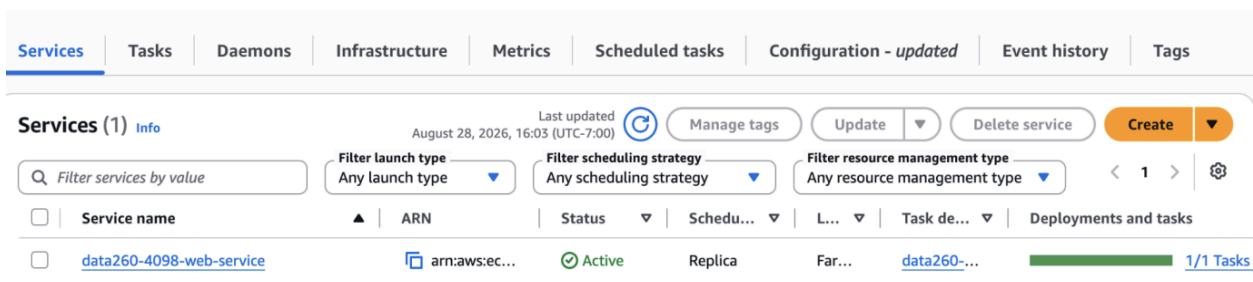


Figure 4. ECS service with one running Fargate task (1/1).

2.3 Public AWS Output

The running task received a public IPv4 address. Loading that address over HTTP displayed the same application served by the ECS task. The service was later scaled to zero desired tasks to stop compute charges; the screenshot remains the deployment evidence.

The screenshot shows a web browser window with the address bar displaying 'Not Secure' and the IP address '50.17.140.49'. The webpage has a light blue background and a central white form titled 'Municipal Transit Incident Report' in bold blue text. The form contains the following elements:

- Incident Title:** A text input field with the placeholder 'Enter a short incident title'.
- Route or Transit Line:** A text input field with the placeholder 'Enter the affected route or transit line'.
- Submitter Email:** A text input field with the placeholder 'Enter your email address'.
- Incident Description:** A large text area with the placeholder 'Describe the transit incident in detail'.
- Incident Category:** A dropdown menu with the placeholder 'Choose a category'.
- Terms and Conditions:** A checkbox labeled 'I agree to the terms and conditions.'.
- Submit Button:** A blue button labeled 'Submit Transit Incident'.

Figure 5. Public webpage served from AWS ECS at the task public IP address.

3. Part 2 - Planner, Reviewer, and Finalizer

In this part, the same title and content pass through three steps. First, the Planner suggests three tags and a short summary. Next, the Reviewer checks whether the tags are relevant and whether the summary follows the word limit. Finally, the Finalizer prepares the result as JSON. I also used Python functions to clean the model response and make sure it contains exactly three tags and a summary of no more than 25 words. All model requests go through `src/model_client.py`.

Pipeline design

| Stage | Primary job | Required output |
|----------|-------------------------------------|-----------------------------|
| Planner | Generate candidate tags and summary | 3 topical tags + 1 sentence |
| Reviewer | Validate relevance and | Cleaned tags/summary + |

| | constraints | issues |
|-----------|-------------------------------------|----------------------------|
| Finalizer | Apply review and finalize | Strict JSON |
| Coercion | Enforce deterministic schema guards | Exactly 3 tags; <=25 words |

I used separate stages so I could see each agent's result before the final output was saved.

Agent call and schema enforcement

```

27
28 def strip_code_and_md(s: str) -> str:
29     """
30     TODO: Remove markdown/code artifacts from model output.
31     Suggested:
32     - remove fenced code blocks
33     - remove inline backticks
34     - normalize whitespace
35     """
36     # Placeholder implementation:
37     text = str(s)
38     text = re.sub(r"```(?:[A-Za-z0-9_+~]~)?", "", text)
39     text = text.replace("`", "")
40     return " ".join(text.split())
41
42
43 def extract_json_block(text: str) -> str:
44     cleaned = strip_code_and_md(text)
45
46     start = cleaned.find("{")
47     end = cleaned.rfind("}")
48
49     if start != -1 and end != -1 and end > start:
50         return cleaned[start:end + 1]
51
52     return json.dumps({"message": cleaned})
53
54

```

```

57
58 def tokens(txt: str) -> List[str]:
59     """
60     TODO: Tokenize into lowercase words (optionally keep hyphens) , filter junk, etc.
61     """
62     return re.findall(r"[a-z]+(?:-[a-z]+)*", str(txt).lower())
63
64
65 def ngrams(words: List[str], n: int) -> Iterable[Tuple[str, ...]]:
66     """
67     TODO: Yield word n-grams from a token list.
68     """
69     for i in range(max(0, len(words) - n + 1)):
70         yield tuple(words[i:i + n])

```

```

72
73 def phrase_candidates(title: str, content: str, maxn: int = 12) -> List[str]:
74     """
75     TODO: Build tag candidates derived ONLY from title+content.
76     Suggested approach:
77     - tokenize + remove STOP words
78     - gather bigrams/trigrams
79     - rank by frequency
80     - fall back to unigrams
81     - return up to maxn
82     """
83
84     title_words = [word for word in tokens(title) if word not in STOP]
85     content_words = [word for word in tokens(content) if word not in STOP]
86
87     counts = {}
88
89     for words in (title_words, content_words):
90         for n in (3, 2):
91             for group in ngrams(words, n):
92                 phrase = " ".join(group)
93                 counts[phrase] = counts.get(phrase, 0) + 1
94
95     ranked_phrases = sorted(
96         counts,
97         key=lambda phrase: (-counts[phrase], phrase)
98     )
99
100     all_words = title_words + content_words
101     unigram_counts = {}
102
103     for word in all_words:
104         unigram_counts[word] = unigram_counts.get(word, 0) + 1
105
106     ranked_words = sorted(
107         unigram_counts,
108         key=lambda word: (-unigram_counts[word], word)
109     )
110
111     candidates = []

```

```

112
113     for candidate in ranked_phrases + ranked_words:
114         if candidate not in candidates:
115             candidates.append(candidate)
116
117         if len(candidates) == maxn:
118             break
119
120     return candidates
121
122

```

```

125 # -----
126 # Output schema coercion
127 # -----
128
129 def coerce_reply(raw_obj: Any, title: str, content: str, strict: bool) -> Dict[str, Any]:
130     """
131     TODO: Coerce arbitrary model output into the required schema:
132     {
133         "thought": str,
134         "message": str (non-empty, <= 60 words),
135         "data": {
136             "tags": [str, str, str],          # exactly 3 topical tags
137             "summary": str,                    # <= 25 words, ends with '.'
138             "issues": [str, ...]
139         }
140     }
141
142     strict=True suggestion:
143     | - enforce at least two multi-word tags
144     """
145
146     obj = raw_obj if isinstance(raw_obj, dict) else {}
147
148     data = obj.get("data", {})
149     if not isinstance(data, dict):
150         data = {}
151
152     candidates = phrase_candidates(title, content)
153     source_words = set(tokens(f"{title} {content}"))
154
155     raw_tags = data.get("tags", [])
156     if not isinstance(raw_tags, list):
157         raw_tags = []
158
159     valid_tags = []
160
161     for tag in raw_tags:
162         cleaned_tag = " ".join(tokens(strip_code_and_md(tag)))
163         tag_words = cleaned_tag.split()
164

```

```

164
165         if (
166             tag_words
167             and all(word in source_words for word in tag_words)
168             and cleaned_tag not in valid_tags
169         ):
170             valid_tags.append(cleaned_tag)
171
172     tags = []
173
174     if strict:
175         for tag in valid_tags + candidates:
176             if len(tag.split()) >= 2 and tag not in tags:
177                 tags.append(tag)
178
179             if len(tags) == 2:
180                 break
181
182     for tag in valid_tags + candidates:
183         if tag not in tags:
184             tags.append(tag)
185
186         if len(tags) == 3:
187             break
188
189     tags = tags[:3]
190
191     summary = strip_code_and_md(data.get("summary", ""))
192
193     if not summary:
194         summary = strip_code_and_md(content)
195
196     summary = re.sub(r"\.{2,}", ".", summary)
197     summary = re.split(r"(?<=[.!?])\s+", summary)[0]
198     summary_words = summary.split()[:25]
199
200     if summary_words:
201         summary = " ".join(summary_words).rstrip(".,;:!?") + "."
202     else:
203         summary = "Summary unavailable."
204

```

```

205     message = strip_code_and_md(obj.get("message", ""))
206
207     if not message:
208         message = "Proposal reviewed; tags and summary prepared."
209
210     message = " ".join(message.split()[:60])
211
212     thought = strip_code_and_md(obj.get("thought", ""))
213
214     raw_issues = data.get("issues", [])
215     if not isinstance(raw_issues, list):
216         raw_issues = [raw_issues] if raw_issues else []
217
218     issues = [
219         strip_code_and_md(issue)
220         for issue in raw_issues
221         if strip_code_and_md(issue)
222     ]
223
224     return {
225         "thought": thought,
226         "message": message,
227         "data": {
228             "tags": tags,
229             "summary": summary,
230             "issues": issues,
231         },
232     }
233
234
235 def parse_and_coerce(text: str, title: str, content: str, strict: bool) -> Dict[str, Any]:
236     """
237     TODO:
238     - extract_json_block()
239     - json.loads()
240     - coerce_reply()
241     - handle JSON parse failures gracefully
242     """

```

```

243     try:
244         obj = json.loads(extract_json_block(text))
245     except Exception:
246         obj = {"message": strip_code_and_md(text)}
247     return coerce_reply(obj, title, content, strict)
248
249
250 # -----
251 # Agent wrapper
252 # -----
253
254 @dataclass
255 class SimpleAgent:
256     name: str
257     system: str
258     model_name: str
259     base_url: str
260     temperature: float
261
262     def respond(
263         self,
264         conversation: List[Dict[str, str]],
265         task: str,
266         title: str,
267         content: str,
268         strict: bool,
269     ) -> Dict[str, Any]:
270         history_text = "\n".join(
271             f'{message["role"]}: {message["content"]}'
272             for message in conversation
273         ) or "(empty)"
274
275         human_instruction = (
276             f"Task:\n{task}\n\n"
277             f"Conversation so far:\n{history_text}\n\n"
278             "Return ONLY one JSON object. Do not use code fences, "
279             "markdown, or explanations outside the JSON. "
280             "Use these keys: thought (string), message "
281             "(non-empty and no more than 60 words), data.tags "
282             "(exactly 3 topical tags), data.summary "
283             "(one sentence of no more than 25 words), and "
284             "data.issues (array)."
285         )

```

```

286
287
288     messages = [
289         {
290             "role": "system",
291             "content": self.system,
292         },
293         {
294             "role": "user",
295             "content": human_instruction,
296         },
297     ]
298
299     result = complete(
300         messages,
301         model=self.model_name,
302         base_url=self.base_url,
303         temperature=self.temperature,
304         response_format="json",
305     )
306
307     return parse_and_coerce(
308         result["content"],
309         title,
310         content,
311         strict,
312     )

```

3.1 Agent Execution Evidence

Exact command

```

python code/agents_demo.py --model qwen3:8b \
--title "Bus Delay at Santa Clara Station" \
--content "The VTA Route 22 bus arrived thirty minutes late at Santa Clara Station and affected passengers during the afternoon commute." \
--email "kazisaminamaraj.mumu@sjsu.edu" \
--temperature 0.0 --strict

```

```

--- Planner (36816 ms) ---
{
  "thought": "Identifying key topics from the blog post to create relevant tags and a concise summary.",
  "message": "The VTA Route 22 bus delay at Santa Clara Station disrupted afternoon commuters by 30 minutes, highlighting public transit reliability concerns.",
  "data": {
    "tags": [
      "clara station",
      "santa clara",
      "santa clara station"
    ],
    "summary": "VTA Route 22 bus delay at Santa Clara Station disrupted afternoon commuters by 30 minutes.",
    "issues": []
  }
}

```

Figure 6. Planner output: three tags, a concise message, and a one-sentence summary

```

--- Reviewer (32263 ms) ---
{
  "thought": "Identified key topics: public transit reliability, specific route impact, and commuter disruption. Ensured tags are specific and summary is concise.",
  "message": "Tags and summary generated based on the blog's focus on VTA Route 22 delay effects. No issues detected.",
  "data": {
    "tags": [
      "vta route delay",
      "clara station",
      "santa clara"
    ],
    "summary": "VTA Route 22 bus delay at Santa Clara Station disrupted afternoon commuters by 30 minutes, raising concerns about public transit reliability.",
    "issues": []
  }
}

```

Figure 7. Reviewer output: topicality and formatting review.

```

Finalized Output
{
  "thought": "Finalizing tags and summary based on the blog's focus on VTA Route 22 delay effects.",
  "message": "Tags and summary reflect the blog's focus on transit delays and commuter impact. No issues detected.",
  "data": {
    "tags": [
      "vta route",
      "clara station",
      "santa clara"
    ],
    "summary": "VTA Route 22 bus delay at Santa Clara Station disrupted afternoon commuters by 30 minutes.",
    "issues": []
  }
}

```

Figure 8. Finalized strict JSON output.

3.2 Part 2 Answers

Q1. What are the final tags?

- vta route
- santa clara
- clara station

Q2. What is the final summary?

VTA Route 22 bus delay at Santa Clara Station disrupted afternoon commuters by 30 minutes.

Q3. Did the Reviewer change anything?

Yes. The Reviewer changed the Planner's first tag from santa clara station commute to vta route delay, making it more directly related to the affected route. It retained the concise summary.

Explanation

| Stage | Responsibility | Output guard |
|-----------|---|---------------------------------------|
| Planner | Propose tags and summary from title and content | Exactly 3 topical tags |
| Reviewer | Check relevance, clarity, and format | Summary <= 25 words |
| Finalizer | Apply feedback and return final result | Strict JSON; issues=[] |
| Coercion | Clean and validate arbitrary model output | Fallback tags and normalized sentence |

| | | |
|-----------|---|-------------------|
| Publisher | Package input, transcript, final result, and UTC date | Serializable JSON |
|-----------|---|-------------------|

Publish-package structure

```

365     transcript: List[Dict[str, str]] = []
366
367     # Planner
368     t0 = time.time()
369     a = planner.respond(transcript, task, args.title, args.content, args.strict)
370     t1 = time.time()
371     transcript.append({"role": "Planner", "content": a.get("message", "")})
372     print(f"\n--- Planner ({int((t1 - t0) * 1000)} ms) ---\n{json.dumps(a, indent=2)}")
373
374     # Reviewer
375     t0 = time.time()
376     b = reviewer.respond(transcript, task, args.title, args.content, args.strict)
377     t1 = time.time()
378     transcript.append({"role": "Reviewer", "content": b.get("message", "")})
379     print(f"\n--- Reviewer ({int((t1 - t0) * 1000)} ms) ---\n{json.dumps(b, indent=2)}")
380
381     # Finalizer
382     final = finalizer.respond(transcript, task, args.title, args.content, args.strict)
383     print(f"\n Finalized Output \n{json.dumps(final, indent=2)}")
384
385     # Publish package
386     package = {
387         "title": args.title,
388         "email": args.email,
389         "content": args.content,
390         "agents": {"transcript": transcript, "final": final.get("data", {})},
391         "submissionDate": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
392     }
393     print(f"\n Publish Package \n{json.dumps(package, indent=2)}")
394
395

```

4. Part 3 - Measuring Non-Determinism

A fixed JSON input was used unchanged for all 40 runs. The experiment executed 20 runs at temperature 0.7 and 20 runs at temperature 0.0. Tag-set comparisons ignored tag order, and latency percentiles used linear interpolation.

Fixed input

```

{ "title": "Bus Delay at Santa Clara Station",
  "content": "The VTA Route 22 bus arrived thirty minutes late at Santa Clara Station and affected
passengers during the afternoon commute.",
  "email": "kazisaminamaraj.mumu@sjsu.edu" }

```

Automation logic

```
37 def main():
38     case_data = json.loads(INPUT_PATH.read_text(encoding="utf-8"))
39
40     if OUTPUT_PATH.exists():
41         results = json.loads(OUTPUT_PATH.read_text(encoding="utf-8"))
42     else:
43         results = []
44
45     completed_runs = {
46         (float(item["temperature"]), int(item["run"]))
47         for item in results
48     }
49
50     temperatures = [0.7, 0.0]
51
52     for temperature in temperatures:
53         for run_number in range(1, 21):
54             run_key = (temperature, run_number)
55
56             if run_key in completed_runs:
57                 print(
58                     f"Skipping completed run: "
59                     f"temperature={temperature}, run={run_number}",
60                     flush=True,
61                 )
62                 continue
63
64             command = [
65                 sys.executable,
66                 str(CODE_DIR / "agents_demo.py"),
67                 "--model",
68                 "qwen3:8b",
69                 "--title",
70                 case_data["title"],
71                 "--content",
72                 case_data["content"],
73                 "--email",
74                 case_data["email"],
75                 "--temperature",
76                 str(temperature),
77                 "--strict",
78             ]
```

```

79
80     print(
81         f"Starting temperature={temperature}, "
82         f"run={run_number}/20",
83         flush=True,
84     )
85
86     start_time = time.perf_counter()
87
88     completed = subprocess.run(
89         command,
90         capture_output=True,
91         text=True,
92     )
93
94     latency_ms = round(
95         (time.perf_counter() - start_time) * 1000
96     )
97
98     if completed.returncode != 0:
99         print(completed.stderr, file=sys.stderr)
100         print(
101             "The experiment stopped, but completed runs "
102             "remain saved. Run this script again to resume.",
103             file=sys.stderr,
104         )
105         sys.exit(completed.returncode)
106
107     marker = "Publish Package"
108
109     if marker not in completed.stdout:
110         print(completed.stdout, file=sys.stderr)
111         raise RuntimeError(
112             "Publish Package was not found in the output."
113         )
114
115     publish_text = completed.stdout.rsplit(marker, 1)[1].strip()
116     publish_package = json.loads(publish_text)
117     final_data = publish_package["agents"]["final"]

```

```

118
119         result = {
120             "temperature": temperature,
121             "run": run_number,
122             "tags": final_data["tags"],
123             "summary": final_data["summary"],
124             "latency_ms": latency_ms,
125             "timestamp_utc": datetime.now(
126                 timezone.utc
127             ).isoformat(),
128         }
129
130         results.append(result)
131         save_results(results)
132
133         print(
134             f"Completed temperature={temperature}, "
135             f"run={run_number}/20, latency={latency_ms} ms",
136             flush=True,
137         )
138
139     print(
140         f"All {len(results)} runs are saved in {OUTPUT_PATH}",
141         flush=True,
142     )

```

Reproduction command

caffeinate -i python code/run_nondeterminism.py | tee -a reports/hw01/RUN_LOG.txt

```
● ((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 % tail -6 reports/hw01/RUN_LOG.txt
Completed temperature=0.0, run=18/20, latency=95678 ms
Starting temperature=0.0, run=19/20
Completed temperature=0.0, run=19/20, latency=95544 ms
Starting temperature=0.0, run=20/20
Completed temperature=0.0, run=20/20, latency=95557 ms
All 40 runs are saved in /Users/samina/Documents/data260-4098/reports/hw01/raw/nondeterminism_runs.json
○ ((.venv) ) (base) samina@Kazis-MacBook-Pro-2 data260-4098 % █
```

Figure: Completion of the 40-run non-determinism experiment.

The script saves each completed run immediately and resumes without repeating completed temperature/run pairs. Raw results are stored in reports/hw01/raw/nondeterminism_runs.json.

4.1 Non-Determinism Results

| Metric | Temperature 0.7 | Temperature 0.0 |
|-------------------------|---------------------------|---------------------------------------|
| Runs | 20 | 20 |
| Distinct tag sets | 9 | 1 |
| Tags in all 20 runs | None | clara station; santa clara; vta route |
| Tags in exactly one run | santa clara station delay | None |

End-to-end pipeline latency

| Percentile | Temperature 0.7 | Temperature 0.0 |
|------------|-----------------|-----------------|
| p50 | 95,109 ms | 95,638 ms |
| p95 | 151,817 ms | 96,491 ms |
| p99 | 181,604 ms | 99,873 ms |

Interpretation

When I used temperature 0.7, the same input produced nine different tag sets across the 20 runs. This means two users could receive different tags even if they submit the same information. At temperature 0.0, all 20 runs produced the same tag set, so the result was more consistent. Some variation is acceptable when the model is only suggesting descriptive tags because several tags may correctly describe one incident. However, variation would not be acceptable when deciding an incident's severity or emergency priority because an inconsistent result could affect the response.

Evidence locations

- Raw 40-run results: reports/hw01/raw/nondeterminism_runs.json
- Console log: reports/hw01/RUN_LOG.txt
- Calculated tables: reports/hw01/METRICS.md

5. Part 4 - Shared Model Adapter and Token Accounting

Both the multi-agent pipeline and interactive code-review client use a single reusable Ollama adapter. The adapter accepts messages, optional tools, model/base URL/temperature controls, an optional response format, and returns content plus input, output, and total token counts.

Reusable adapter - src/model_client.py

```
src > model_client.py
1  import os
2  from typing import Any, Dict, List, Optional
3
4  from ollama import Client
5
6
7  DEFAULT_MODEL = os.environ.get("SMOL_MODEL", "qwen3:8b")
8  DEFAULT_BASE_URL = os.environ.get(
9      "OLLAMA_URL",
10     "http://localhost:11434",
11 )
12
13
14 def complete(
15     messages: List[Dict[str, str]],
16     tools: Optional[List[Dict[str, Any]]] = None,
17     *,
18     model: Optional[str] = None,
19     base_url: Optional[str] = None,
20     temperature: Optional[float] = None,
21     response_format: Optional[str] = None,
22 ) -> Dict[str, Any]:
23     selected_model = model or DEFAULT_MODEL
24     selected_base_url = base_url or DEFAULT_BASE_URL
25
26     if temperature is None:
27         temperature = float(
28             os.environ.get("MODEL_TEMPERATURE", "0.0")
29         )
30
31     client = Client(host=selected_base_url)
32
33     request = {
34         "model": selected_model,
35         "messages": messages,
36         "stream": False,
37         "options": {
38             "temperature": temperature,
39         },
40     }
41
```

```

42     if tools:
43         request["tools"] = tools
44
45     if response_format:
46         request["format"] = response_format
47
48     response = client.chat(**request)
49
50     input_tokens = int(response.prompt_eval_count or 0)
51     output_tokens = int(response.eval_count or 0)
52
53     tool_calls = []
54
55     for tool_call in response.message.tool_calls or []:
56         tool_calls.append(tool_call.model_dump())
57
58     return {
59         "content": response.message.content or "",
60         "tool_calls": tool_calls,
61         "input_tokens": input_tokens,
62         "output_tokens": output_tokens,
63         "total_tokens": input_tokens + output_tokens,
64         "model": response.model,
65     }

```

Bullet-only system instructions - AGENT.md

```

↓ AGENT.md > abc # Code Review Agent Instructions
1  # Code Review Agent Instructions
2
3  - Review submitted code for correctness, security, performance, and readability.
4  - Respond using bullet points only.
5  - Begin every response line with ` - `.
6  - Do not use headings, paragraphs, numbered lists, or code fences.
7  - Keep each bullet concise and specific.
8  - Provide no more than five bullets.
9  - If there are no problems, respond exactly with ` - No issues found.`

```

5.1 Interactive Client Implementation

The client maintains conversation history, resends it on every model call, validates the bullet-only format, records each turn, supports /stats after any turn, prints cumulative statistics on /exit, and writes the structured evidence to token_counts.json.

```

code > hw1_client.py
33
34 def main():
35     system_prompt = AGENT_PATH.read_text(encoding="utf-8")
36
37     messages = [
38         {
39             "role": "system",
40             "content": system_prompt,
41         }
42     ]
43
44     turn_count = 0
45     cumulative_input_tokens = 0
46     cumulative_output_tokens = 0
47
48     token_log = {
49         "turns": [],
50         "stats_snapshots": [],
51     }
52
53     print("Code Review Client")
54     print("Enter code or a review request.")
55     print("Commands: /stats and /exit")
56
57     while True:
58         try:
59             user_input = input("\nYou: ").strip()
60         except (EOFError, KeyboardInterrupt):
61             print()
62             break
63
64         if not user_input:
65             continue
66
67         if user_input.lower() in {"/exit", "/quit"}:
68             break
69
70         if user_input.lower() == "/stats":
71             snapshot = {
72                 "after_turn": turn_count,
73                 "turn_count": turn_count,
74                 "cumulative_input_tokens": cumulative_input_tokens,
75                 "cumulative_output_tokens": cumulative_output_tokens,
76                 "history_length": history_length(messages),
77             }
78

```

```

78
79         token_log["stats_snapshots"].append(snapshot)
80         save_token_log(token_log)
81
82         print(
83             "Stats: "
84             f"turns={turn_count}, "
85             f"cumulative_input_tokens="
86             f"{cumulative_input_tokens}, "
87             f"cumulative_output_tokens="
88             f"{cumulative_output_tokens}, "
89             f"history_length="
90             f"{snapshot['history_length']}"
91         )
92         continue
93
94     messages.append(
95         {
96             "role": "user",
97             "content": user_input,
98         }
99     )
100
101     result = complete(messages)
102     assistant_text = result["content"].strip()
103
104     messages.append(
105         {
106             "role": "assistant",
107             "content": assistant_text,
108         }
109     )
110
111     turn_count += 1
112     cumulative_input_tokens += result["input_tokens"]
113     cumulative_output_tokens += result["output_tokens"]
114
115     response_lines = [
116         line
117         for line in assistant_text.splitlines()
118         if line.strip()
119     ]
120

```

```

120
121     bullet_only = bool(response_lines) and all(
122         | line.startswith("- ")
123         | for line in response_lines
124     )
125
126     turn_record = {
127         "turn": turn_count,
128         "input_tokens": result["input_tokens"],
129         "output_tokens": result["output_tokens"],
130         "total_tokens": result["total_tokens"],
131         "cumulative_input_tokens": cumulative_input_tokens,
132         "cumulative_output_tokens": cumulative_output_tokens,
133         "history_length": history_length(messages),
134         "bullet_only": bullet_only,
135     }
136
137     token_log["turns"].append(turn_record)
138     save_token_log(token_log)
139
140     print(f"\nAssistant:\n{assistant_text}")
141     print(
142         | "Bullet-only format followed: "
143         | f"{'YES' if bullet_only else 'NO'}"
144     )
145     print(
146         | "Turn tokens: "
147         | f"input={result['input_tokens']}, "
148         | f"output={result['output_tokens']}, "
149         | f"total={result['total_tokens']}"
150     )
151
152     save_token_log(token_log)
153
154     print("\nCumulative statistics")
155     print(f"Turns: {turn_count}")
156     print(
157         | f"Cumulative input tokens: "
158         | f"{cumulative_input_tokens}"
159     )
160     print(
161         | f"Cumulative output tokens: "
162         | f"{cumulative_output_tokens}"
163     )

```

```

164
165
166     if __name__ == "__main__":
167         | main()

```

5.2 Five-Turn Conversation Evidence

You: Review this Python code: `def add(a, b): return a + b`

Assistant:

- No issues found.

Bullet-only format followed: YES

Turn tokens: input=112, output=935, total=1047

You: Review this Python code: `def divide(a, b): return a / b`

Assistant:

- Potential `ZeroDivisionError` if ``b`` is zero; no error handling.

- Function name ``divide`` implies division, but the code only performs ``a / b`` (correct).

- No type checking or input validation.

- No docstring for clarity.

- No handling for non-numeric inputs.

Bullet-only format followed: YES

Turn tokens: input=142, output=497, total=639

Figure 9. Turns 1 and 2 with per-turn token counts and bullet-only checks.

You: Review this Python code: `password = "admin123"`

Assistant:

- Hardcoded password stored in plain text; insecure for production use.

- Uses a weak, default `password` ("admin123") susceptible to brute-force attacks.

- No encryption or hashing applied to the password.

- No environment variable or secure vault integration for sensitive data.

- Exposes password risk if code is accessed by unauthorized users.

Bullet-only format followed: YES

Turn tokens: input=231, output=357, total=588

You: `/stats`

Stats: turns=3, cumulative_input_tokens=485, cumulative_output_tokens=1789, history_length=1463

Figure 10. Turn 3 followed by the required `/stats` snapshot.

You: Review this Python code: `items = []; [items.append(i) for i in range(1000000)]`

Assistant:

- The list comprehension ``[items.append(i) for i in ...]`` creates a list of ``None`` values (since ``append`` returns ``None``), not the intended list of numbers.

- Inefficient use of list comprehension; direct assignment ``items = [i for i in range(1000000)]`` is clearer and more performant.

- Redundant use of ``items = []`` followed by a list comprehension that overwrites it; unnecessary complexity.

- The semicolon at the end is unnecessary and may indicate a syntax error or formatting issue.

- Poor readability due to the misleading list comprehension structure.

Bullet-only format followed: YES

Turn tokens: input=339, output=740, total=1079

Figure 11. Turn 4 review and token counts.

5.3 Final Turn and Exit Statistics

```
You: Review this Python code: query = f"SELECT * FROM users WHERE name = '{name}'"

Assistant:
- Vulnerable to SQL injection due to direct string interpolation; use parameterized queries instead.
- Lack of input validation or sanitization for the `name` variable.
- No error handling for invalid or malicious input.
- Risk of broken SQL syntax if `name` contains unescaped single quotes.
- Hardcoded SQL structure reduces flexibility and security.
Bullet-only format followed: YES
Turn tokens: input=503, output=448, total=951

You: /stats
Stats: turns=5, cumulative_input_tokens=1327, cumulative_output_tokens=2977, history_length=2698

You: /exit

Cumulative statistics
Turns: 5
Cumulative input tokens: 1327
Cumulative output tokens: 2977
((.venv) ) (base) samina@Kazis-MacBook-Pro-2 code %
```

Figure 12. Turn 5, second /stats snapshot, /exit, and cumulative totals.

Per-turn token accounting

| Turn | Input | Output | Total | Cum. input | Cum. output | History | Bullet |
|------|-------|--------|-------|------------|-------------|---------|--------|
| 1 | 112 | 935 | 1,047 | 112 | 935 | 588 | Yes |
| 2 | 142 | 497 | 639 | 254 | 1,432 | 985 | Yes |
| 3 | 231 | 357 | 588 | 485 | 1,789 | 1,463 | Yes |
| 4 | 339 | 740 | 1,079 | 824 | 2,529 | 2184 | Yes |
| 5 | 503 | 448 | 951 | 1,327 | 2,977 | 2,698 | Yes |

5.4 Part 4

Why is prior conversation context resent with every turn?

Each model request is separate, so the model does not automatically remember the earlier conversation. The program sends the system instructions and previous messages again so the model can understand what has already happened.

How is a system prompt different from a user message?

A system prompt gives the model its overall role and the rules it should follow. A user message contains the specific question, code, or task that the user wants the model to handle.

Why do input tokens grow over a conversation?

The input-token count grows because every new request includes more conversation history. As the user and assistant add messages, more text must be sent back to the model during the next turn.

What eventually limits that growth?

The model has a maximum context-window size, so the conversation history cannot grow forever. When the limit is approached, the application may need to remove older messages, summarize them, or keep only the most important information.

Required /stats snapshots

| After turn | Turn count | Cumulative input | Cumulative output | History length |
|------------|------------|------------------|-------------------|----------------|
| 3 | 3 | 485 | 1,789 | 1,463 |
| 5 | 5 | 1,327 | 2,977 | 2,698 |

All five responses passed the strict bullet-only validation.

6. Verification and AI Use Disclosure

Overall verification status: PASS

| Automated check | Result | Details |
|---------------------|--------|---|
| required_files | PASS | All required files exist. |
| python_version | PASS | Active Python version: 3.12 |
| html_form | PASS | Required form fields and script link found. |
| model_adapter_usage | PASS | Agent pipeline and CLI use |

| | | |
|---------------------|------|--|
| | | src/model_client.py. |
| nondeterminism_runs | PASS | Total=40, temp_0.7=20,
temp_0.0=20 |
| token_accounting | PASS | Turns=5, input_total=1327,
output_total=2977,
stats_after=[3, 5] |

AI use disclosure: ChatGPT

I used ChatGPT when I needed help understanding some of the assignment instructions and working through unfamiliar Docker and AWS steps. It also helped me troubleshoot errors and gave me suggestions for organizing parts of the code. I personally edited the project files, ran the commands, configured Docker and AWS, completed all 40 model runs, checked the results, and took the screenshots. One problem I encountered was that the starter code used the old langchain_community ChatOllama import, which did not work in my environment. I was also using Python 3.13, while the assignment required Python 3.11 or 3.12. I identified these problems from the error messages and by checking the assignment requirements. To fix them, I created a Python 3.12 virtual environment and used a reusable model adapter in src/model_client.py. After making these changes, the agent pipeline and interactive client ran successfully, and the verification checks passed.

The complete disclosure is stored in reports/hw01/AI_USE.md.

Final evidence checklist

| Artifact | Status | Repository location |
|----------------------|----------|---|
| Local Docker webpage | Included | reports/hw01/raw/local_docker_webpage.png |
| ECR upload | Included | reports/hw01/raw/ecr_image_uploaded.png |
| ECS task running | Included | reports/hw01/raw/aws_ecs_task_running.png |
| Public AWS webpage | Included | reports/hw01/raw/aws_ecs_public_webpage.png |
| Part 2 outputs | Included | reports/hw01/raw/part2_*.png |

| | | |
|------------------------|----------|---|
| 40-run raw results | Included | reports/hw01/raw/nondeterminism_runs.json |
| Five-turn transcript | Included | reports/hw01/raw/part4_*.png |
| Token accounting | Included | reports/hw01/raw/token_counts.json |
| Automated verification | Pass | reports/hw01/verification.json |